TSCIM: A 28nm Transposed Stochastic CIM Macro for On-Chip Training and Inference

Yu Liu[a,b], Yang Lou[a], Kangkang Mao[a], Xin Li[a,b], Chenghu Dai[a,b], Xiulong Wu[a,b], Zhiting Lin[a,b*]
[a] Anhui University, Hefei, China
[b] The Anhui High-performance Integrated Circuit Engineering Research Center, Hefei, China
[*] Corresponding Author Email: ztlin@ahu.edu.cn

2025 IEEE International Symposium on Circuits and Systems (ISCAS) | 979-8-3503-5683-0/25/$31.00 ©2025 IEEE | DOI: 10.1109/ISCAS56072.2025.11044117

***Abstract*—This work introduces a novel Transposed Stochastic Computing-in-Memory (TSCIM) macro designed to enhance the efficiency of on-chip training and inference. The macro incorporates a novel stochastic quantization strategy and utilizes a transposed separated wordline SRAM to enable multi-bit signed MAC operations. Furthermore, a stochastic adder tree is utilized to minimize area and power consumption overhead. The design includes a 4Kb SRAM CIM macro implemented in 28 nm CMOS technology. Simulation results show that the power consumption of the stochastic accumulation circuit (SAC) is reduced by 63.6%, while the area overhead is decreased by a factor of 7.73 compared to designs using full adder (FA) adder trees. Additionally, the computation latency is decreased by 16× compared to traditional stochastic circuits. The TSCIM macro can achieve a peak energy efficiency of 63.02 TOPS/W and an area efficiency of 15.54 TOPS/mm^2.**

***Index Terms*—Computing-in-memory, Stochastic computing, Quantization, Transpose, On-chip training.**

I. Introduction

With the rapid advancement of artificial intelligence (AI) technologies, edge computing devices are playing an increasingly pivotal role in intelligent applications. Computing-in-memory (CIM) has emerged as a promising solution for accelerating AI computations [1]. CIM is categorized into digital and analog types, as illustrated in Fig. 1 (a) and (b). Red, blue and green backgrounds denote larger, medium and smaller overheads of area and power consumption respectively.

Digital CIM offers higher precision, it uses adder tree to achieve partial sum accumulation, which takes up most of the area and power consumption overhead [2]. Attempts have been made to mitigate these overheads by employing approximate computation and sparsity techniques, but the adder tree overhead remains substantial [3], [4]. Stochastic computing (SC), another approximate computation paradigm, facilitates multiplication and addition (MAC) using simplistic logic gates [5]. In SC, numbers are represented by probabilities, the probability of "1" in the bitstream represents the numerical value. Yang [6] proposed a digital Stochastic-CIM, utilizing the inherent "AND logic" of 10T cells for multiplication and "wired-OR" for addition, substantially reducing the adder tree overhead. However, storing stochastic binary streams in arrays results in exponential storage overhead, thereby compromising storage density. Furthermore, multi-bit stochastic computation inherently leads to exponential computation times. The excessively long stochastic bit streams significantly reduce the throughput of CIM operations.

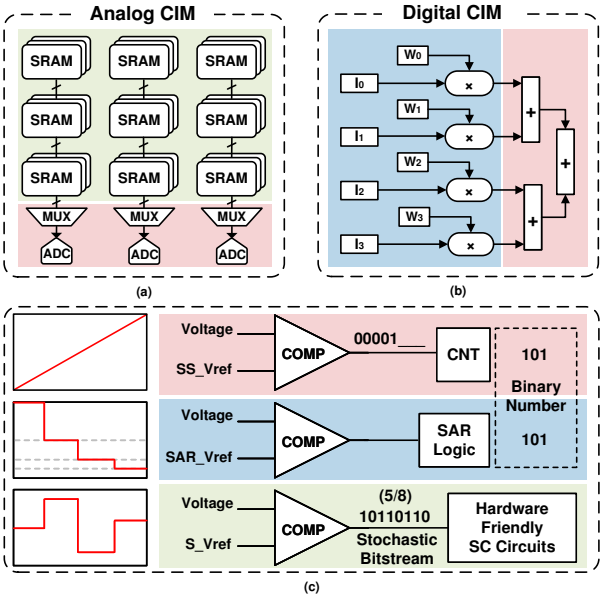


Fig. 1. (a) Analog CIM architecture. (b) Digital CIM architecture. (c) Different quantization strategies of SS, SAR and stochastic methods.

In contrast, analog CIMs generally feature reduced power consumption, yet they contend with ADC and DAC overheads [1]. The ADC area is typically large, necessitating multiplexing across multiple columns in an SRAM array, which disrupts CIM's parallelism. The ±CIM [7] employs commercial dual-port 8T to perform signed computations across two BLs but fails to fully leverage the complementary nature of SRAM. Additionally, the accuracy of its computed results is sensitive to variations in process, voltage, and temperature [8]. Furthermore, earlier works concentrate mainly on inference and lack support for on-chip training [9], [10], inadequately meeting the needs of AI edge devices.

To address these challenges, we propose a novel digital-analog hybrid architecture supporting both training and inference. The advantages of the proposed architecture are as follows:

This work was supported by University Synergy Innovation Program of Anhui Province under Grant GXXT-2023-003, National Natural Science Foundation of China under Grant 62202003, National Natural Science Foundation of China under Grant 62104001

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:10:56 UTC from IEEE Xplore. Restrictions apply.

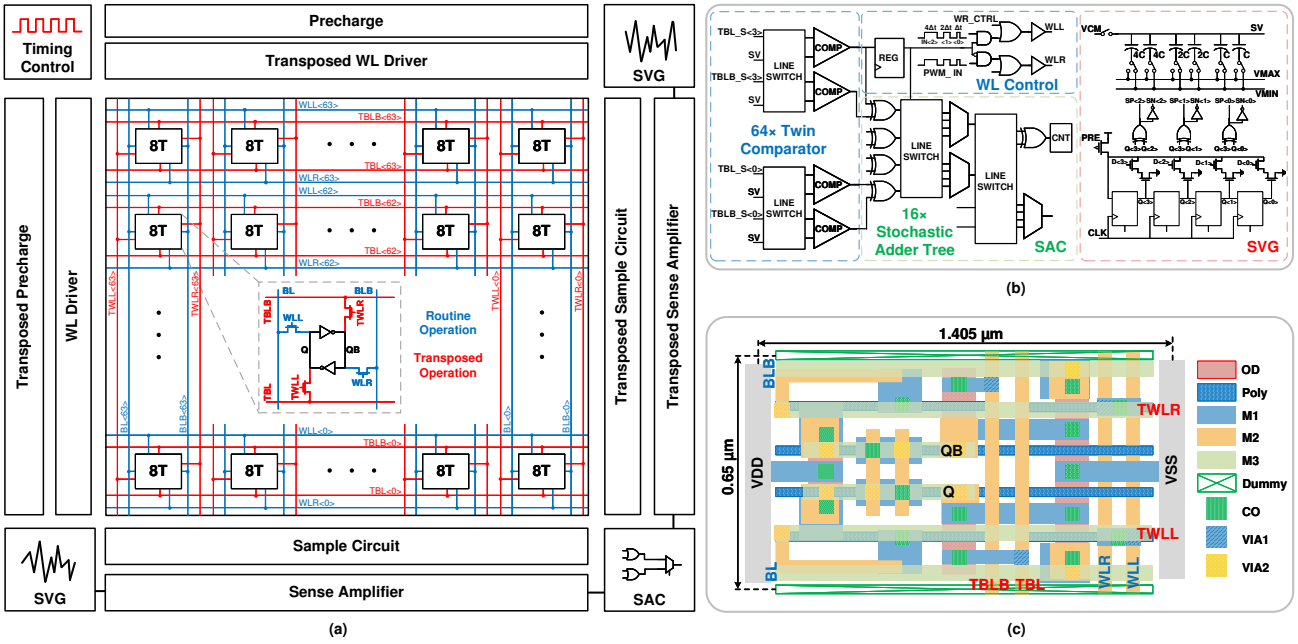


Fig. 2. (a) Overall architecture and schematic of 8T cell. (b) Stochastic number generation and stochastic computing circuit. (c) 8T SRAM layout (90° rotated).

1) Distinct from traditional quantization methods such as Single Slope (SS) and Successive Approximation Register (SAR), the quantization method in TSCIM compares BL's voltage with stochastic voltage, quantizing the BL's voltage drop into stochastic binary streams, as illustrated in Fig. 1 (c). Subsequent additions of partial sums occur within a stochastic computation framework.

2) A transposed separated wordline (WL) structure is designed for the CIM macro to facilitate signed and multi-bit number computations, supporting variable precision operations such as INT4 and INT8. Moreover, the sampling capacitor uses MOM capacitors positioned above the array diagonally, allowing for reuse both horizontally and vertically without incurring additional area overhead.

3) The 8-bit data is divided into 4-bit segments for computation, with partial sums recombined using stochastic computation. The weighted summation of results is performed stochastically, significantly reducing the exponential latency inherent in stochastic computation.

The remainder of this paper is organized as follows. Section II describes the proposed CIM architecture. Section III provides an analysis of the simulation results. Finally, Section IV concludes the paper.

II. Proposed Architecture

A. Overall Architecture and Bitcell Circuit

Fig. 2 (a) illustrates the overall architecture of the proposed TSCIM macro, which consists of a 64-row by 64-column SRAM array, two sets of transposed read and write circuits, two sets of stochastic voltage generator (SVG), and a stochastic accumulation circuit (SAC). The TSCIM can be operated in both routine and transposed modes in parallel. During routine computation operations, transposed operations can be simultaneously executed for updating weights, thereby enhancing efficiency. Fig. 2 (a) also shows the transposed separated-WL 8T SRAM cell. This cell primarily comprises a 4T latch and a 4T transposed switching transistor. The WLL and WLR are used for routine operations, while the transposed WL (TWLL and TWLR) are used for transposed operations.

During inference tasks, activations are fed horizontally through WLL and WLR, and accumulated vertically through the BL and BLB. Conversely, for training tasks, loss values are fed vertically through the TWLL and TWLR, and accumulated horizontally through the TBL and TBLB. During the inference CIM operations, one multi-bit weight is mapped horizontally across the array. The sign bit of the weight is pre-read horizontally to determine which side of the WL should be activated: if the weight is positive, WLL is activated; if negative, the WLR is activated. Fig. 2 (b) illustrates the WL driver circuit, in read/write mode, PWM_IN is set to 0, and the WLs are controlled by WR_CTRL. In CIM mode, WR_CTRL is set to 0, and the multi-bit input is fed as pulse width modulation (PWM). During CIM operations, the WL is underdriven to extend the discharge time.

Fig. 2 (c) presents the layout of the proposed 8T cell, rotated by 90°. The bit-cell area is 0.91325 μm^2 with the logic rule of 28 nm. Adjacent bitcells are placed in proximity using a shared dummy poly, which allows these bitcells to be tightly coupled, thus reducing the area overhead.

B. Stochastic Computing Circuit

Fig. 2 (b) illustrates an example circuit where analog values are converted into stochastic bitstreams to perform stochastic computation addition. Each SRAM column is equipped with a

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:10:56 UTC from IEEE Xplore. Restrictions apply.

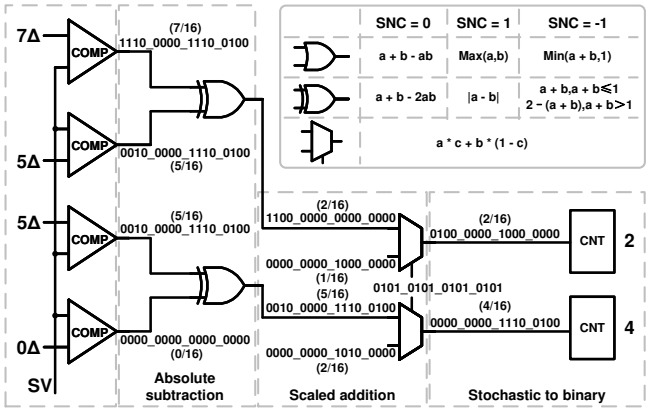


Fig. 3. Example of Stochastic computing using 4-bit quantization and stochastic computation logic with different SNC.

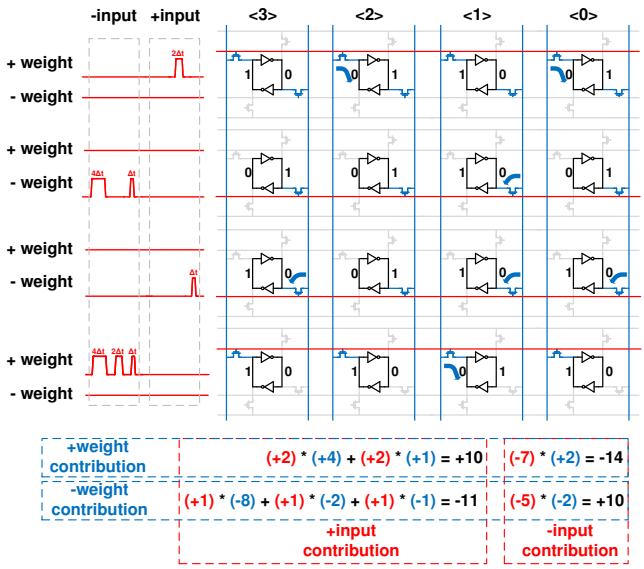


Fig. 4. Example of 4 bit × 4 bit MAC operation with signed input and signed weight.

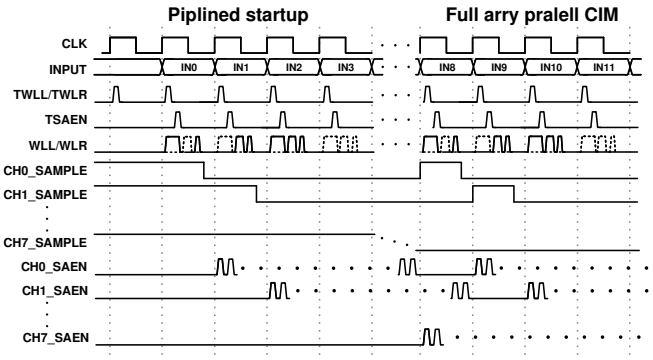


Fig. 5. Timing diagram.

pair of comparators. In read/write mode, only one comparator is activated to read data. In CIM mode, BL and BLB are connected to two comparators, which compare them against a stochastic voltage (SV). The SV is generated by a stochastic voltage generator (SVG), controlled by the output of a linear feedback shift register (LFSR), which switches the split capacitor capacitive digital-to-analog converter (CDAC). The seed of the LFSR can be modified via control inputs D<3:0>, allowing the generation of stochastic numbers with varying correlations. Unlike successive approximation register (SAR) ADCs, which require each ADC to have a dedicated CDAC, multiple comparators can share a single SV in this approach, significantly reducing power and area overheads.

The computational logic in SC is influenced by the stochastic number correlation (SNC), as illustrated in Fig. 3. An OR operation on two datasets can be used to implement approximate addition when SNC is zero [11]. However, without ensuring low sparsity and uncorrelation, there can be significant deviations in computation results [12]. In contrast, a MUX can perform scaled stochastic addition without requiring control over the correlation of multiple input datasets, as long as the correlation between the selector and inputs is zero [13]. By controlling the select line, both weighted summation and normalized summation can be achieved. Extending this capability to an N-to-1 MUX, the computation logic can be expressed as $P(\text{out}) = \frac{1}{N}\sum_{i=1}^{N} P_i(\text{in})$.

By combining the characteristics of MUX and XNOR gates, we propose quantizing positive and negative numbers using an SVG with a correlation of one and utilizing a stream with a correlation of zero as the select input of the MUX gate to perform scaled stochastic addition. As shown in Fig. 2 (c), every four pairs form a group that is used to perform shift accumulation operations with weights. The positive and negative bitstreams are processed separately. The resulting positive and negative bitstreams can be configured to produce binary outputs via XOR gates and counters or continue with new stochastic additions, thus forming a stochastic adder tree.

C. Full Arry Parallel CIM Operation

Fig. 4 shows a 4bit × 4bit operation with signed input and signed weight. To maintain symmetry between positive and negative numbers, the weights are stored in the array using inverse code. Fig. 5 shows the timing diagram of the MAC operation. The sign bits are initially read horizontally by control signals TWLL, TWLR and TSAEN. Subsequently, the WLL/WLR is toggled based on the input value and the sign bit of the weight. Once the input process is complete, the sampling capacitor switch, CH_SAMPLE, is pulled down, and the MAC result is stored in the sampling capacitor. The comparator then conducts multiple comparisons to convert the voltage value into a stochastic bit stream. During inference, the weights are divided into multiple channels. After the horizontal pre-reading operation of one channel, the process is repeated for the next channel. After the piplined startup is done, the MAC operation can be executed in full array parallel.

While the array is performing MAC operations, SAC can accumulate stochastic bit streams by conducting weighted or

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:10:56 UTC from IEEE Xplore. Restrictions apply.

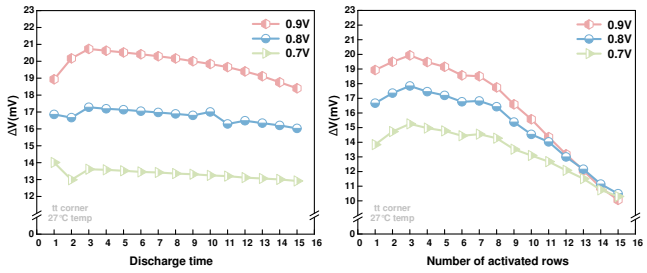


Fig. 6. Impact of discharge time and number of activated rows on BL voltage variations under different WL supply voltages.

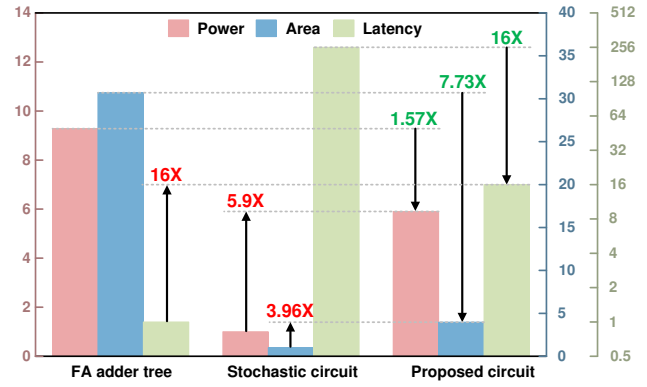


Fig. 7. The normalized area, power consumption and latency of the SAC with the FA adder tree design and traditional stochastic design.

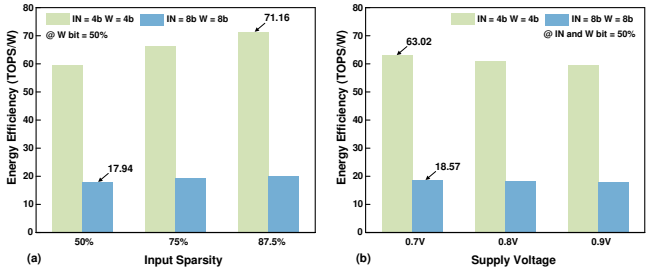


Fig. 8. (a) The energy efficiency results of the proposed CIM macro with different input sparsity. (b) The energy efficiency of the proposed CIM macro at different supply voltages.

unweighted summation of partial sums based on the weight mapping relationship. The final stochastic bit stream sum is then converted into a binary number output through a counter.

III. Experimental Results

The TSCIM macro is designed in a 28 nm process. We extrac the parasitic parameters from the layout, then use Cadence Virtuoso and AMS simulator to obtain the result of the simulation.

Fig. 6 illustrates the linearity of various WL driver voltages. When the WL is underdriven with a 0.7 V supply voltage, The BL voltage swings between 900 mV and 600 mV and gets well-quantized linearity.

TABLE I Comparison With Prior Art

	This work	VLSI 2024 [11]	A-SSCC 2022 [6]	JSSC 2021 [7]	JSSC 2024 [14]
Technology	**28nm**	12nm	65nm	40nm	28nm
Cell type	**8T**	8T	10T	8T	8T
MAC operation	**AMS**	Digital	Digital	Analog	Analog
Quantization strategy	**4b Stochastic**	1b SA	1b SA	6b SAR	4b Flash
Frequency (MHz)	**500**	600	4	65	100
Array size	**4Kb**	230Kb	520Kb	32Kb	16Kb
Voltage	**0.9V (CIM) 0.7V-0.9V (SAC)**	0.64V-0.85V	0.7V-1.05V	1.1V	0.7V-1.1V
Peak energy efficiency (TOPS/W)	**63.02 (I:4b/W:4b)**	46 (I:6b/W:6b)	7.96 (I:8b/W:8b)	41 (I:2b/W:2b)	50.1 (I:4b/W:8b)
Area efficiency (TOPS/mm^2)	**15.54 (I:4b/W:4b)**	6.8 (I:6b/W:6b)	N/A	2.4 (I:2b/W:2b)	2.86 (I:4b/W:8b)

Fig. 7 illustrates the normalized area, power consumption and latency of the proposed SAC with the FA adder tree design and traditional stochastic design. Notably, the Y-axis for latency is on an exponential scale. Experimental results show that SAC achieves a 1.57× reduction in power consumption and a 7.73× reduction in area compared to the FA adder tree design, as well as a 16× reduction in time consumption compared to traditional stochastic designs. However, the proposed SAC requires more processing time than the FA adder tree and results in greater power consumption and area overhead compared to the traditional stochastic computation stochastic.

Fig. 8 (a) illustrates the energy efficiency at different input sparsity while keeping weight sparsity at 50%. Analog core supply is set to 0.9 V, WL is driven by 0.7 V. Fig. 8 (b) illustrates the energy efficiency at digital stochastic computing circuits are supplied with different voltages. When the bit width of the input and weight are both 4 bits, the peak energy efficiency is 63.02 TOPS/W at 0.7 V supply voltage. When the bit width of both the input and weight are 8 bits, the peak energy efficiency is 18.57 TOPS/W.

The performances of the previous methods and the CIM architecture proposed in this paper are compared in Table I. The area efficiency can reach 15.54 TOPS/mm^2, which is attributed to the compact array layout, fully parallel CIM operation mode, and the minimal SAC area overhead. In conclusion, the proposed architecture provides higher energy efficiency and better area efficiency compared to existing CIM architectures.

IV. Conclusion

This work proposes a digital-analog hybrid CIM architecture that supports both training and inference. The designed circuit employs an innovative stochastic quantization method, and the proposed SAC effectively combines the advantages of both binary and stochastic computation, achieving a balance between the two. Additionally, the implementation of efficient timing logic ensures optimal array performance. Experimental results demonstrate that the proposed CIM macro exhibits excellent energy efficiency and outstanding area utilization, facilitating the efficient training and inference of neural networks.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:10:56 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] Z. Sun, S. Kvatinsky, X. Si et al., "A full spectrum of computing-in-memory technologies," Nat Electron, vol. 6, pp. 823–835, 2023. doi: 10.1038/s41928-023-01053-4.

[2] D. Wang, C. -T. Lin, G. K. Chen, P. Knag, R. K. Krishnamurthy and M. Seok, "DIMC: 2219TOPS/W 2569F2/b Digital In-Memory Computing Macro in 28nm Based on Approximate Arithmetic Hardware," 2022 IEEE International Solid-State Circuits Conference (ISSCC), San Francisco, CA, USA, 2022, pp. 266-268, doi: 10.1109/ISSCC42614.2022.9731659.

[3] Y. He et al., "An RRAM-Based Digital Computing-in-Memory Macro With Dynamic Voltage Sense Amplifier and Sparse-Aware Approximate Adder Tree," in IEEE Transactions on Circuits and Systems II: Express Briefs, vol. 70, no. 2, pp. 416-420, Feb. 2023, doi: 10.1109/TCSII.2022.3209872.

[4] C. He et al., "LSAC: A Low-Power Adder Tree for Digital Computing-in-Memory by Sparsity and Approximate Circuits Co-Design," in IEEE Transactions on Circuits and Systems II: Express Briefs, vol. 71, no. 2, pp. 852-856, Feb. 2024, doi: 10.1109/TCSII.2023.3304752.

[5] C. Lammie, J. K. Eshraghian, W. D. Lu and M. R. Azghadi, "Memristive Stochastic Computing for Deep Learning Parameter Optimization," in IEEE Transactions on Circuits and Systems II: Express Briefs, vol. 68, no. 5, pp. 1650-1654, May 2021, doi: 10.1109/TCSII.2021.3065932.

[6] J. Yang, T. Li, W. Romaszkan, P. Gupta and S. Pamarti, "A 65nm 8-bit All-Digital Stochastic-Compute-In-Memory Deep Learning Processor," 2022 IEEE Asian Solid-State Circuits Conference (A-SSCC), Taipei, Taiwan, 2022, pp. 10-11, doi: 10.1109/A-SSCC56115.2022.9980613.

[7] S. Jain, L. Lin and M. Alioto, "±CIM SRAM for Signed In-Memory Broad-Purpose Computing From DSP to Neural Processing," in IEEE Journal of Solid-State Circuits, vol. 56, no. 10, pp. 2981-2992, Oct. 2021, doi: 10.1109/JSSC.2021.3092759.

[8] Z. Lin et al., "Cascade Current Mirror to Improve Linearity and Consistency in SRAM In-Memory Computing," in IEEE Journal of Solid-State Circuits, vol. 56, no. 8, pp. 2550-2562, Aug. 2021, doi: 10.1109/JSSC.2021.3063719.

[9] J. -W. Su et al., "16.3 A 28nm 384kb 6T-SRAM Computation-in-Memory Macro with 8b Precision for AI Edge Chips," 2021 IEEE International Solid-State Circuits Conference (ISSCC), San Francisco, CA, USA, 2021, pp. 250-252, doi: 10.1109/ISSCC42613.2021.9365984.

[10] S. Wang, Y. Ma and Y. Kang, "AFT-CIM: An Energy Efficient ADC-Free Transpose Computing-in-Memory Macro for MAC Operations," 2024 IEEE International Symposium on Circuits and Systems (ISCAS), Singapore, Singapore, 2024, pp. 1-5, doi: 10.1109/ISCAS58744.2024.10558546.

[11] J. Yang, A. Graening, W. Romaszkan, V. K. Jacob, P. Gupta and S. Pamarti, "A 278-514M Event/s ADC-Less Stochastic Compute-In-Memory Convolution Accelerator for Event Camera," 2024 IEEE Symposium on VLSI Technology and Circuits (VLSI Technology and Circuits), Honolulu, HI, USA, 2024, pp. 1-2, doi: 10.1109/VLSITechnologyandCir46783.2024.10631484.

[12] Z. Lin, G. Xie, S. Wang, J. Han and Y. Zhang, "A Review of Deterministic Approaches to Stochastic Computing," 2021 IEEE/ACM International Symposium on Nanoscale Architectures (NANOARCH), AB, Canada, 2021, pp. 1-6, doi: 10.1109/NANOARCH53687.2021.9642242.

[13] A. Alaghi, Cheng Li and J. P. Hayes, "Stochastic circuits for real-time image-processing applications," 2013 50th ACM/EDAC/IEEE Design Automation Conference (DAC), Austin, TX, USA, 2013, pp. 1-6, doi: 10.1145/2463209.2488901.

[14] K. Lee, J. Kim and J. Park, "A 28-nm 50.1-TOPS/W P-8T SRAM Compute-In-Memory Macro Design With BL Charge-Sharing-Based In-SRAM DAC/ADC Operations," in IEEE Journal of Solid-State Circuits, vol. 59, no. 6, pp. 1926-1937, June 2024, doi: 10.1109/JSSC.2023.3334566.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:10:56 UTC from IEEE Xplore. Restrictions apply.